

摘要：

GPT-6 Astra发布引爆AI安全争议——“循环Transformer会让推理过程变得不可见”的说法，令业界人心惶惶。OpenAI首席科学家亲自下场辟谣，而技术大V Sebastian Raschka则用一篇深度长文拆解真相：思维链变短，不过是更强模型犯错更少、无需大量草稿纸的自然结果，与架构无关。

过去两周里，「循环深度」这个词变成了 AI 安全讨论的焦点。

起因是 OpenAI 新旗舰模型 GPT-6 Astra 发布前两天，The Information 援引内部消息称，Astra 用上了一种叫 recurrent depth（循环深度）、也就是 looped transformer（循环 Transformer）的设计，而这种设计「会让模型的部分乃至全部推理过程变得不可见」

OpenAI 表示，其即将推出的AI模型Astra在编程和在计算机上操作应用程序等能力方面迈上了一个新台阶。但据一位了解Astra开发情况的人士称，一项提升该模型性能的创新技术也意味着，该模型以及其他类似模型将更少暴露其“思考”过程，从而使其更难被监控是否存在不良行为迹象。

虽然这一局限对Astra来说未必是重大问题，但这项技术已在OpenAI内部以及整个行业引发担忧：采用并强化这一技术的AI开发者，是否会难以防范那种最近入侵了OpenAI自身系统以及Hugging Face等其他公司系统的失控AI。

要点

- OpenAI即将推出的Astra模型采用了一种名为“循环深度”的技术，这种技术可以掩盖AI的推理过程
- 行业研究人员担心，这项技术如果应用不当，可能会妨碍防范失控AI的努力
- OpenAI表示，它将能够监控Astra，以迅速发现任何不当行为

技术支持：[Deep Research](#)

OpenAI正在使用的这项新技术被称为循环深度或循环transformer，它允许AI模型通过多次处理同一文本，来改进其答案。

与市面上最先进的模型不同——这些模型在完成任务前会以文字形式展示它们如何“思考”一项任务——这项新技术的工作方式会遮蔽AI的部分或全部推理过程，也就是所谓的“思维链”。这意味着，模型为完成任务所采取的步骤无法轻易被人类读取或理解。

据知情人士透露，OpenAI已限制Astra中循环深度技术的使用，以确保模型仍能生成可读的思维链，公司研究人员也能充分监控其推理过程。（OpenAI 周二在博客中表示，将推出具备“额外思维链监控以快速检测和遏制”潜在不当行为的Astra。）

周二晚间，OpenAI首席科学家Jakub Pachocki在X上发帖称，尽管对模型思维链的监控“脆弱”且“不幸地正朝着负面方向发展”，但他希望劝阻整个行业竞相开发那些不产生AI研究人员理解模型行为所需的那种清晰推理的模型。

模型正式上线后，争议进一步升温，OpenAI 首席科学家 Jakub Pachocki 亲自出面澄清，说他想避免「一场由混乱的报道引发的、奔向不可监控性的竞赛」，并强调包括 Astra 在内的当前前沿模型，其计算图深度与 GPT-4 的差距仍在两倍以内。



Jakub Pachocki
@merettm



我想防止因混乱的报道而引发一场奔向不可监控性的竞赛。我们当前前沿模

在一片语焉不详的转述中，著名技术博主、《Build a Large Language Model (From Scratch)》的作者 Sebastian Raschka 写了一篇长文，把技术细节摊开讲了一遍。

他先给出自己使用 Astra 的观察，包括它在 3D 渲染和图形任务上不成比例的领先、通过 Codex/ChatGPT 应用直接操作本地软件的能力，以及 OpenAI 采购数万台 Mac 作为强化学习环境背后的训练逻辑。随后他从 2018 年的 Universal Transformer 讲起，一路梳理到今年的开放权重模型 Nanbeige4.2-3B、字节跳动的 Ouro 和谷歌的 Mixture-of-Recursions.....

而对于最受关注的那个问题，他给出了否定回答。在他看来，循环 Transformer 并不是思维链变短、变得难以监控的原因，更合理的解释是能力更强的模型犯错更少、回溯更少，因此不需要那么长的草稿纸——GPT-5.6 家族里 Luna 比 Sol 多用 80% 的 token 才达到相近性能，没有人因此说 Sol 的可解释性更差。Astra 系统卡确实承认推理轨迹的可监控性出现了退步，但没有证据表明可以把这笔账记在循环架构头上。文章最后还介绍了几项新研究。

以下是博客全文：

GPT-6 Astra, Looped Transformers, and

博客地址：<https://magazine.sebastianraschka.com/p/gpt-6-astra-looped-transformers-and>

过去几周发生了很多事。我相信眼下大家最关心的，还是 OpenAI 的 GPT-6 Astra。尤其是它的性能表现、其中的循环 Transformer / 循环深度设计，以及那个说 Astra 在「隐藏」自己推理轨迹（也就是思维链）的传言。

所以在这篇文章里，我想先谈谈对 Astra 的一些初步印象，以及我对这一切走向的看法。然后，我会详细讲清楚什么是「循环 Transformer」，以及它和隐藏思维链究竟有没有关系（或者说，到底有没有关系）。

最后，在讲完循环 Transformer 的基础之后，我想介绍几篇近期论文带来的新洞见。

1. GPT-6 Astra 使用印象

先说第一件事。在进入架构传言和相关研究文献之前，我先简单总结一下我对 GPT-6 Astra 的一些观察和零散发现。



上周，OpenAI 大张旗鼓地发布了新模型 GPT-6 Astra。过去几天我一直在用它。这是一个极其出色的模型，很可能是截至本文写作时我用过的最好的模型。但它究竟改进了什么？又是怎么做到的？

1.1 Astra 的基准测试表现

Astra 是我目前用过的最好的模型。而且相对于其他模型，它在 3D 渲染和动画任务上好得不成比例。我的意思是，虽然它几乎在所有类别上都甩开了前代 GPT-5.6（写作、数学、编程等等），但在图形类演示上的领先尤其明显。

这一点在基准测试里也有体现。比如下图所示，GPT-6 Astra 在数学和编程上确实很强。

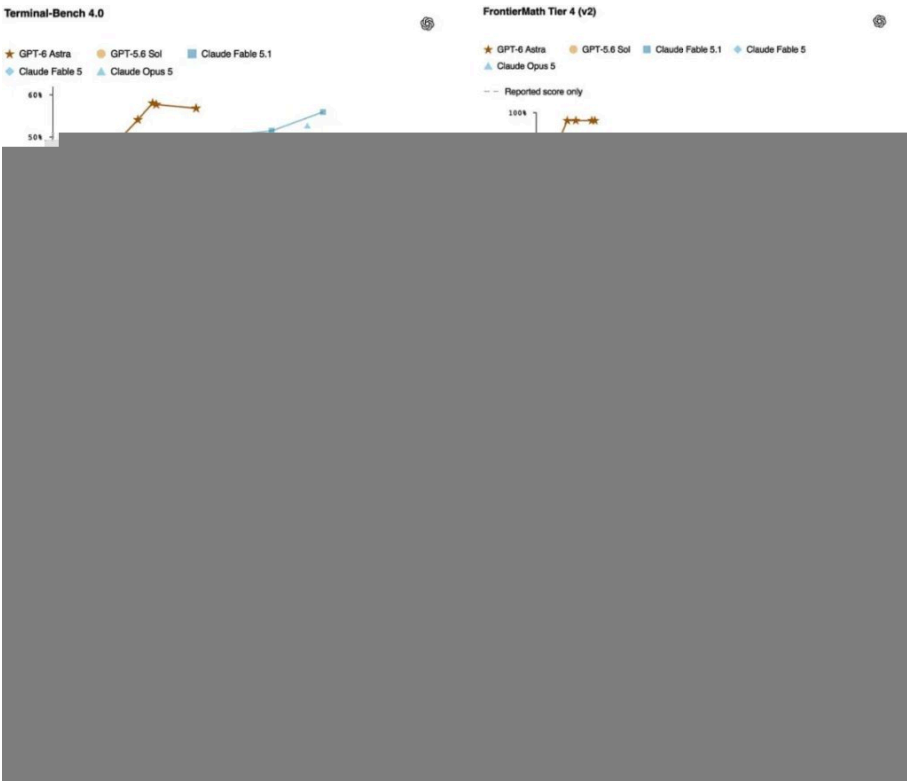


图 1：三个常见编程基准和一个高难度数学基准的结果选摘。更多基准结果见 Astra；发布博客：<https://openai.com/index/gpt-6-astra/>

其中一个亮点（图中未展示）是，Astra 在 ARC-AGI-3 基准上拿到了 99.9%，而 GPT-5.6 Sol 只有 7.8%。这项基准衡量的是逻辑谜题求解与泛化能力的混合表现。不过，数学、编程和计算机操作类基准更值得关注，因为它们更接近真实使用场景。

回到前图右下角的 Artificial Analysis Coding Agent Index v1.4，它把多项智能体编程任务混合在一起。在这项指标上，GPT-6 Astra 明显处在前沿，但并没有大幅甩开对手。下面这张更综合的 Artificial Analysis Intelligence Index 也反映了同样的情况，它混合的是多种类型的任务，而不只是编程任务。



图 2：Artificial Analysis Intelligence Index，来自 <https://artificialanalysis.ai>

Artificial Analysis 这套基准的一大优势在于它是独立的，因此相比模型开发方自评的基准，可能更值得信任。

具体的测试框架（harness）取决于基准本身。例如 GDPval-AA 和 AA-Briefcase 对所有参与对比的 LLM 都统一使用他们自己开源的极简框架 Stirrup。而在上图这版 Intelligence Index v4.2 中，Terminal-Bench v2.1 用的是 Terminus 2， τ^3 -Banking 用的是 τ -Bench 框架。单独的 Coding Agent Index 则还会对比不同的编程智能体框架。

对于使用统一框架的评测来说，这样的比较更接近同一标准下的对照。但与此同时，模型在训练时通常都是围绕某一个主力框架来开发的，对其他框架的微调相对较少。而且，这个主力框架往往本身就是为了契合并放大该模型的长处而设计的。

所以，某些智能体评测可能低估了 Astra 在自家主力框架下的实际表现。至于这对它的 Intelligence Index 得分影响有多大，需要在相同任务上跨框架对比 Astra 才能知道。

顺带说一句：一位同事最近向我建议（Claude Code 负责人也有类似推荐），把你现有的一部分 [AGENTS.md](#) 内容和 [SKILL.md](#) 文件删掉或归档，也许并不是坏事。因为新一代 LLM 在理解提示词、解决手头问题上已经更高效了。额外的手把手指导反而可能不必要地束缚新模型，导致更差的方案。

当然，我不是说以后再也别用 [SKILL.md](#) 文件了。对某些工作流来说，它们能提升复用时的效率，因为模型不必再重新摸索一遍。我想说的是，有些工作流根本不需要描述；而且那些「旧」描述可能已经不再是最优解，LLM 自己或许能想出更好的方案。所以，也许是时候更新或者重新生成这些指令文件了。

1.2 计算机操作能力

GPT-6 Astra 在图像和渲染任务上似乎格外强。当这类任务涉及与图形用户界面交互时，它们同时也展示出了计算机操作（computer use）能力，也就是模型通过 Codex/ChatGPT 应用来操作你本地电脑上的软件。

计算机操作正是这个模型相比其他模型真正出彩的地方。而且任何和图形相关的东西，在社交平台上都很容易做出有趣又直观的演示。网上这类令人印象深刻的例子非常多，从用 Blender 建模渲染纽约市，到虚拟看房导览。



举个例子。下面是一组对比：我让 GPT-6 Astra Medium 和 High 在浏览器版的 MS Paint 里，用我电脑上的鼠标重画一张我的照片（没用 Extra High 和 Max，因为我不想把 token 全耗光 :)）。

这不仅体现了模型的绘画能力，更重要的是体现了它使用你电脑上工具的能力（这里是画图程序，你可以看到模型通过鼠标指针在操作界面）。

这并不是第一个在框架内具备通用计算机操作能力的模型。比如从今年早些时候起，我就成功用 GPT 模型完成过一些 UI 任务，例如 Excel 里的报销相关工作等等。不过，计算机操作仍是一项相对较新的能力，它由框架提供支持，用起来通常感觉还不够成熟。这也说得通：LLM 本质上是文本模型，所以最容易摘的果子自然是写作、编程以及调用 API 和 CLI。

与此同时，还有很多工具和软件（暂时）没有提供 CLI。与其等着谁来设计这个接口，不如直接提升模型使用图形界面的能力（而且如前所述，这本来也能做出漂亮又惊艳的演示）。这在某种程度上类似于当下人形机器人的发展。诚然，在流水线这类已有专用机器的场景里，人形机器人并不是效率最高的选择。但它们胜在通用。

因此我预计，接下来几个月甚至几年，也会是计算机操作能力在 LLM 和智能体框架这两个层面持续打磨的时期。也就是说，除了当前已有的能力、以及在数学和编程上的继续拓展之外，模型的训练会越来越多地把计算机操作考虑进去。这也会让 LLM 对科技圈之外的日常电脑任务更加友好。

1.3 计算机操作能力是怎么训练的

计算机操作这一趋势，也与近期的报道相吻合：OpenAI 采购了数万台 Mac Mini 和 Mac Studio 用于强化学习。这里的 Mac 并不是真的拿来训练模型（那还是用 GPU 更合适），而是在模型训练期间提供 macOS 环境，让模型学会使用这套操作系统以及其中的工具。

那么，在这些 Mac 上的计算机操作训练具体是怎么运作的？简单说，这些 Mac（准确地说是它们的 macOS 系统）充当模型在训练过程中可以交互的环境。

基本流程如下：

- 给模型一个任务作为提示，例如「打开 xyz 应用并完成 abc」。
- 向它提供 macOS 界面的截图（这通常由框架完成）。
- LLM 随后预测鼠标/键盘动作，比如点击、按键、滚动等等。
- 在 Mac 上执行这些动作（同样由框架完成）。
- 执行完上一步的动作后，把更新后环境的新截图喂给模型。
- 重复第 2 到第 5 步，直到任务成功或失败。

- 。把成功/失败信号和验证器（或者说打分器）作为训练反馈，包括在后训练阶段做强化学习；这类类似于常规的可验证奖励强化学习（RLVR）。

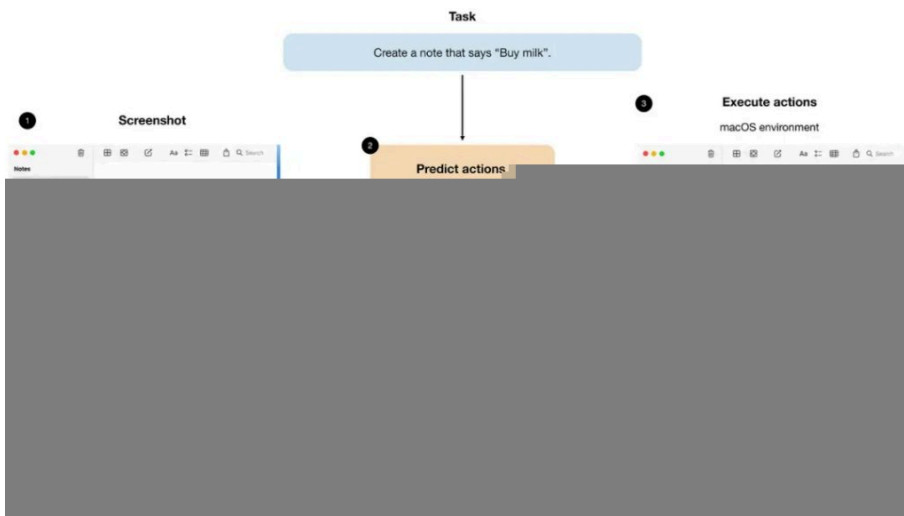


图 3：计算机操作训练流程概览。

再强调一次，这里的 Mac 主要是环境，而不是训练期间运行或更新模型的机器。模型很可能跑在英伟达 GPU 上，通过 API 与这些 Mac 通信。顺带一提，英伟达 CEO 提到，GPT-6 Astra 是在约 10 万块 Grace Blackwell GPU 上训练的。

1.4 GPT-6 Astra 仍然是一个推理模型

上一节讲的对计算机操作训练的侧重，并不意味着训练流程发生了根本性的范式转变。GPT-6 Astra（以及可预见的未来里几乎所有 LLM）仍然是推理模型。也就是说，这类 LLM 用可验证奖励强化学习（RLVR）来训练，并会生成中间的推理轨迹，也就是思维链。

不过关于 GPT-6 Astra 作为推理模型的部分，尤其是隐藏思维链的问题，我会在本文稍后再展开。

2. 循环 Transformer

话说回来，在模型正式发布前大约两天，新闻媒体 The Information 发表了一篇报道称，根据内部消息，Astra 用上了一种叫「循环深度」（recurrent depth）或者「循环 Transformer」（looped transformers）的概念。

“ The new technique OpenAI is using, known as

recurrent depth or looped trans

图 4：来自 The Information 的引述（来源：<https://www.theinformation.com/articles/secret-technique-behind-openais-astra-model-sparks-security-concerns>）

在接下来的几个小节里，我会先解释什么是循环 Transformer，关于隐藏思维链的说法留到本文后面再谈。

（循环 Transformer 这部分讲解可能显得有点长，但我真心认为，它有助于建立对这项技术的基础理解。有了这层理解，我们才能判断「它会遮蔽推理轨迹或思维链」这个说法到底站不站得住脚。）

2.1 复用 Transformer 块

那么，什么是循环 Transformer？

循环 Transformer 本质上是一种架构上的微调，核心思路是让中间表示多次通过同一批 Transformer 块，而不是只过一次。相比单纯堆更多的块，这里的「诀窍」在于：在这些遍历过程中，权重是保持不变的。

定义与术语

本文中我会使用以下几个术语：

- Transformer 块（transformer block）是包含注意力、前馈模块、归一化和捷径连接的一个单元。论文里常把这些块称为「Transformer 层」。
- 栈（stack）指一串 Transformer 块组成的序列。
- 一次块应用（block application）指让输入过一遍某个 Transformer 块。



循环 Transformer 并不新鲜，其基本思路早在 2018 年的 Universal Transformers 论文中就出现过。不过在讨论 Universal Transformer 之前，我们先从一个更简单的例子说起：Nanbeige4.2-3B。这是 7 月发布的一个开放权重 LLM，今年夏天早些时候我在 Substack Notes 和我的 LLM 架构画廊里都介绍过。

下图展示的 Nanbeige 架构，基本上看起来就是个常规 Transformer。但请注意，它多了一条橙色箭头，绕回到 Transformer 栈的开头。

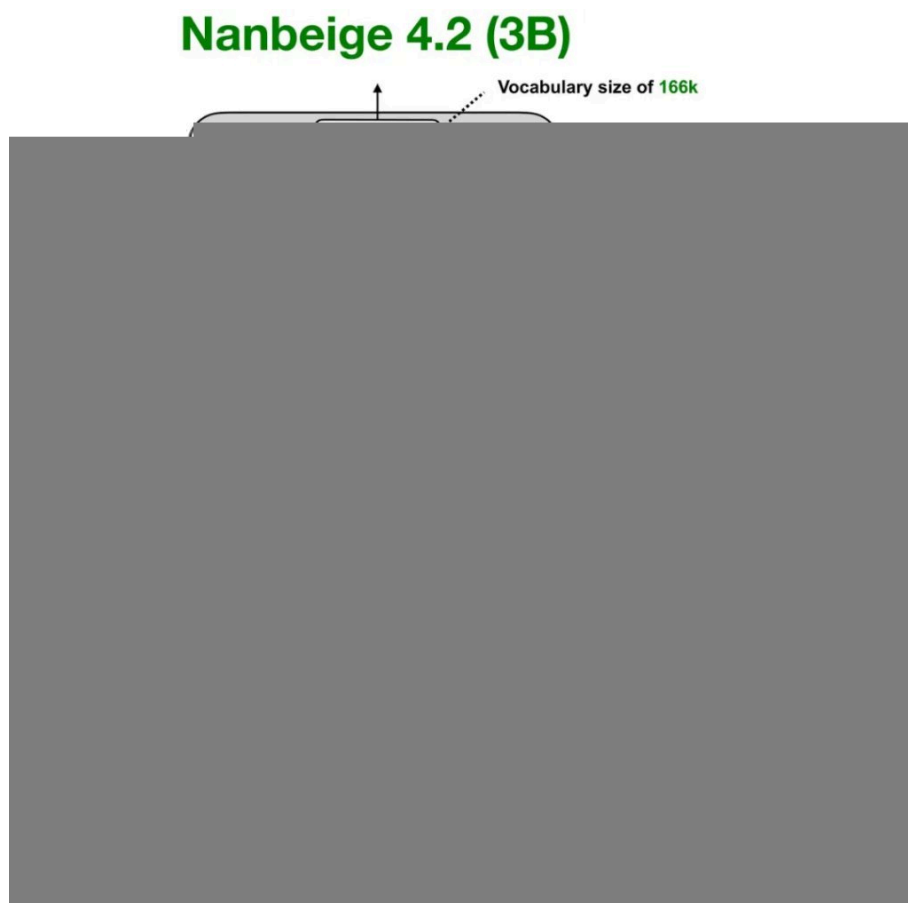


图 5：Nanbeige4.2-3B 把同一组 22 个 Transformer 块应用了两次。橙色箭头标出了中间表示被送回栈中的位置。

我们自下而上走一遍。首先，和其他任何基于 Transformer 的 LLM 一样，输入文本先被 token 化，再转换成嵌入向量。这些向量随后通过 22 个 Transformer 块，而每个块都有自己的一套权重。

循环 Transformer 的特点在于：第一遍走完之后，隐藏状态会被送回这同样的 22 个块再过一遍。于是块 1 被再次应用，接着是块 2，一直到块 22。

如果把这个计算过程展开，就相当于 44 次块应用。但与拥有 44 个独立块的常规 Transformer 不同，第二轮这 22 次块应用复用的是第一轮权重。比如第 23 次块应用用的是块 1 的权重，第 24 次用的是块 2 的权重，依此类推。

图 6：Nanbeige4.2-3B 展开后，相当于让输入两次通过同样的 22 个 Transformer 块，共计 44 次块应用。

所以整个思路就是：把有效深度从 22 次块应用提升到 44 次，而不必新增一套 Transformer 权重。

顺带一问：为什么是 2 轮，而不是 3 轮、4 轮或者更多？Nanbeige 论文里细节不多，但他们表示这基本上就是效率最高的配置。把循环次数从 2 增加到 3 确实能提升建模性能，但额外的计算开销并不划算。

2.2 循环的成本

那么，我们为什么要做这种循环？说到底，它是「靠堆更多 Transformer 块把模型做大」之外的另一种选择。

举例来说，一个把 22 个 Transformer 块用两次的模型，其 Transformer 块的参数量大约只有拥有 44 个常规块的模型的一半。

这样一来，存储权重所需的内存就减少了。补充一句：嵌入层和输出层通常很大，在总参数中占相当比例，但它们不在这项比较范围内。（以 Nanbeige 4.2 3B 为例，嵌入层和输出层约占 3B 总参数的 25%；如果让这两层共享权重，这个比例可以降到 12.5%。

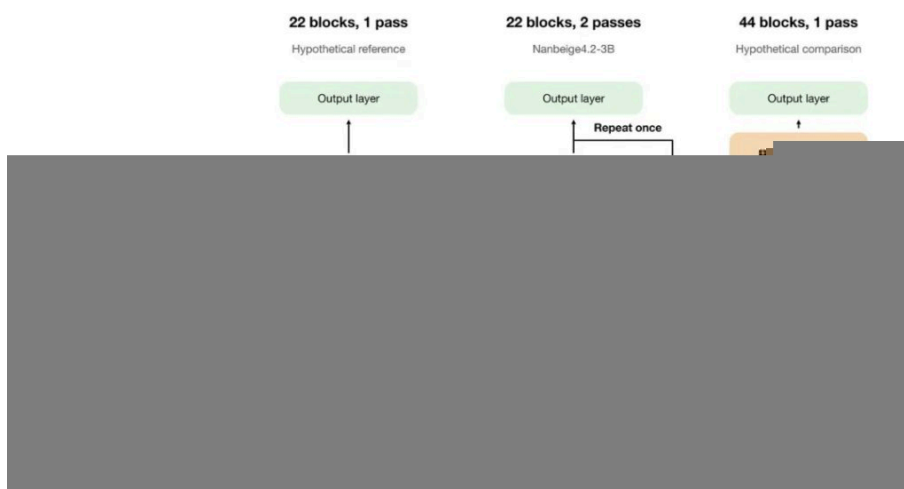


图 7：传统方案与循环方案所需参数量的并排对比。

当然，在循环中复用同样的块仍然是要算的。更准确地说，前向传播时中间输入要走完 44 次块应用。而在训练时，梯度要反向流经共享栈的这两轮重复。所以相比只用一次这 22 个块，这会带来大量额外工作。实际上，其开销和拥有 44 个独立块差不多，区别只在于优化器要更新的独立参数更少，反向传播依然要走全部 44 次块应用。

还有 KV 缓存的问题。无论是常规 Transformer 还是循环 Transformer，KV 缓存都会存下先前 token 的注意力键和值，供后续每一步 token 生成时复用。

尽管循环 Transformer 里存在权重共享，但第二遍进入某个块的中间状态是不一样的。因此在 KV 缓存中，这两轮 Transformer 栈产生的键和值也不同（和不做循环时一样）。所以，KV 缓存这边也省不下什么。

说得更具体些。以第 1 次和第 23 次块应用为例，在循环 Transformer 里它们用的都是块 1，但每一次应用仍然需要各自的 KV 缓存条目。既然两轮都得单独保留缓存，那么这个被重复使用的 22 块栈，其 KV 缓存需求就和一个拥有 44 个独立块的常规 Transformer 一样。

有意思的是，Nanbeige 的研究者在论文中提到，他们试过在两轮之间共享 KV 缓存。这当然把 KV 缓存的体积减半了，但模型表现不如使用独立缓存的版本，也就是他们最终发布的那一版。

在继续看其他循环 Transformer 设计之前，先把 Nanbeige 这部分补完。他们的技术报告还讨论了另外两个选择或者说权衡。

- 从头训练循环架构，效果好于把已经预训练好的 Transformer 通过 upcycling 改造过来。
- 如上一节所述，两轮是他们更偏好的权衡点。更多轮次带来的额外收益很小，却会拖慢训练，也让优化变得更不稳定。

所以，循环轮数是我们必须做的另一个架构选择。如前所述，Nanbeige 把它固定成了 2。但我们也可以让它随 token 而变，接下来就会看到这种做法。

2.3 Universal Transformer 与灵活的循环次数

现在回到 Universal Transformers。在 Nanbeige 中，我们把一个含 22 个 Transformer 块的栈应用两次。而在 2018 年的 Universal Transformer 论文中，被反复应用的是同一个 Transformer 块，而不是一整个栈。不过主要思路是相似的。

另外，步数可以是固定的，但论文也探索了自适应停止（adaptive halting）。比如某个位置上的 token 可能只走一到两轮循环，另一个 token 可能走三到四轮，依此类推。这让模型可以灵活地把算力分配给那些真正能从额外计算中受益的 token。

那么循环次数是怎么定下来的？这里模型用了一个小的、经过训练的函数，在每一步为每个位置输出一个所谓的停止概率。它把这些概率在连续的循环上累加，一旦某个位置的累加值超过阈值，就停止循环。此外还设有一个最大循环次数，以防万一，用来限制计算量。

图 8：Universal Transformer 中的自适应停止机制。

另一个循环 Transformer 的例子是字节跳动的 Ouro。比如 Ouro-Thinking 2.6B 把同一组 48 个 Transformer 块应用了四次。这就是 192 次块应用，而只需存储 48 个独立块的权重。基本上，这比 Nanbeige 更激进。此外，一个学习得到的退出门（exit gate）会给不同的退出点分配概率，再用累计概率的阈值来决定由哪一轮提供输出。所以它也借用了 Universal Transformer 的自适应停止思路，而 Nanbeige 没有用这一套。（不过这里有个实际的注意点：官方发布的 Hugging Face 实现会先算完所有配置好的轮次，再选择输出。所以循环次数看起来实际上被硬编码成了 4。）

2.4 用路由决定循环次数

另一种思路是 Mixture-of-Recursions，这是 2025 年的一篇论文，本质上是前面讲的 Universal Transformer 的进阶版。与 Universal Transformer 类似，单个 token 会一次或多次通过 Transformer 块，如下图所示。但它的创新在于：循环次数是如何按 token 逐个决定的。

在下面这张出自论文的图中，被循环（重复）的那个栈被称为递归块（recursion block）。它包含若干 Transformer 块，并夹在独立的首尾两个 Transformer 块之间，图中标为 Layer 0 和 Layer L-1。

图 9：Mixture-of-Recursions 在不同的 token 位置上，把共享栈应用不同的次数。高亮文字展示了 1 次、2 次或 3 次遍历的例子。图改编自 Mixture-of-Recursions 论文。

模型怎么决定一个 token 该过几次递归块？在前面讨论的 Universal Transformer 里，靠的是每一步学习得到的停止概率。而 Mixture-of-Recursions 这里用的是一个小型的、可学习的路由器。这与混合专家模型中的路由思路类似，只不过这里的路由决策决定的是共享栈要应用多少次。

路由器作用在 token 的隐藏表示上，而隐藏表示里也包含了上下文信息。所以我们不该把它理解成「某个特定 token 每次出现都被分配相同的遍历次数」，也就是说，上图里的「People」并不总是走 3 次循环。这个决定会随着该词出现的位置以及前文内容而变化。

那么路由具体是怎么运作的？论文探索了两种做法，如下图所示。

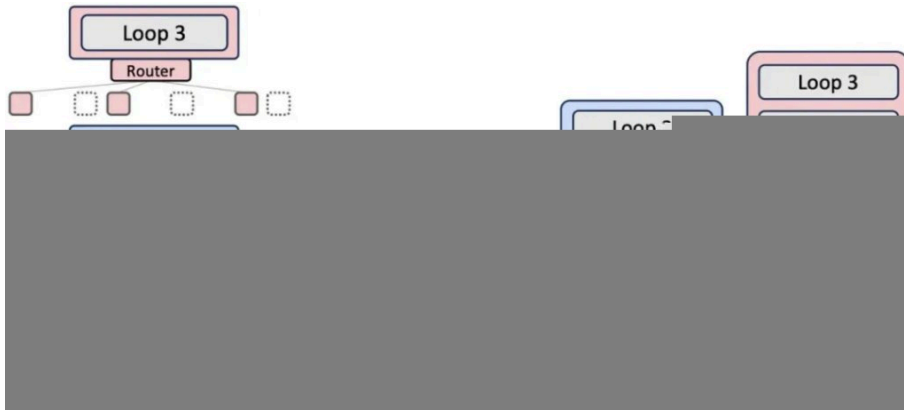


图 10：选择递归深度的两种方式。左边由路由器在每一步选出哪些 token 继续参与；右边由单个路由器在一开始就分配好遍历次数。图出自 Mixture-of-Recursions 论文。

在专家选择路由（expert-choice routing，上图左侧子图）中，每个递归步骤自行选择要处理哪些 token，退出的 token 不再参与后续步骤。在 token 选择路由（token-choice routing，右侧）中，路由器在一开始就做出一次决定，把每个 token 分配到走一次、两次或三次的路径上。

两种情况下，Transformer 权重都会在各轮遍历之间复用，和 Nanbeige 等做法类似。额外的灵活性来自于可以决定每个 token 获得多少计算量。模型和它的路由器是一起训练的，因此模型能在训练中学会应对这些不同的路径。

2.5 效果究竟如何？

下面这张出自 Mixture-of-Recursions 论文的图，对比了常规 Transformer（Vanilla）、固定递归次数的 Transformer（Recursive）和 Mixture-of-Recursions（MoR）在不同模型规模和算力预算（x 轴）下的表现。

图 11：四种模型规模、三种训练算力预算下的验证损失。图出自 Mixture-of-Recursions 论文。

在最小的模型规模上，常规 Transformer 表现最好。到了更大的模型上，Mixture-of-Recursions 追了上来，而且往往表现更好，在训练预算较小时尤其明显。而在最大的预算下，几条曲线非常接近。所以，优势取决于模型规模，也取决于我们在训练上花了多少算力。

另一个细节是：训练算力相同，并不意味着训练 token 数相同。通过跳过部分计算，Mixture-of-Recursions 能在同样的预算内处理更多 token。

我觉得这是个很有意思的例子，因为它说明在循环 Transformer 这个思路内部还有若干选择：每个位置循环多少次，以及这个次数由什么来决定。

简而言之，如果模型足够大，使用循环 Transformer 能在固定算力预算下提升模型质量。（这也说明了做大规模实验的重要性；比如只看 135M 参数的小模型，我们会得出完全相反的结论。）

3. 插叙：循环神经网络（RNN）

顺带一提，如果你有深度学习背景，甚至了解 1990 年代的人工神经网络，那么这种循环、或者说「循环深度」的思路，你应该会觉得有几分眼熟。还记得循环神经网络（RNN）吗？RNN 的整个思路就是复用上一次迭代的层，也就是权重。

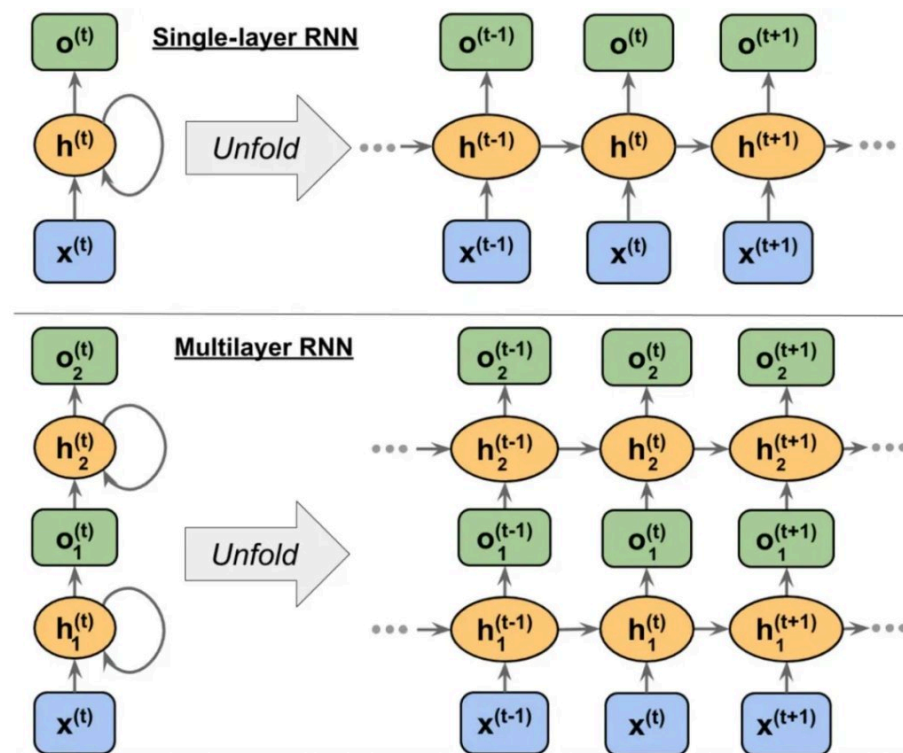


图 12：RNN 示意图，出自我 2022 年的《Machine Learning with PyTorch and Scikit-Learn》一书

主要区别在于，RNN 是跨时间步复用权重。也就是说，隐藏状态从一个 token 传递到下一个 token。而在循环 Transformer 中，一个 token 的循环发生在架构的深度方向上。

换句话说，在常规 RNN 中，每一步接收的是输入序列的下一个元素和上一步的隐藏状态。所以当 RNN 处理一段文本时，它一次读一个词或一个 token，并把前面词的信息通过隐藏状态往前带。

而在循环 Transformer 中，某个 token 的中间表示会多次通过 Transformer 栈。模型仍然靠注意力在 token 之间传递信息。

如果这个类比让你有点晕，也不必太在意。也许更简单的理解方式是：把循环 Transformer 看成对 Transformer 块的复用，类似于把模型做大，只不过权重是共享的。

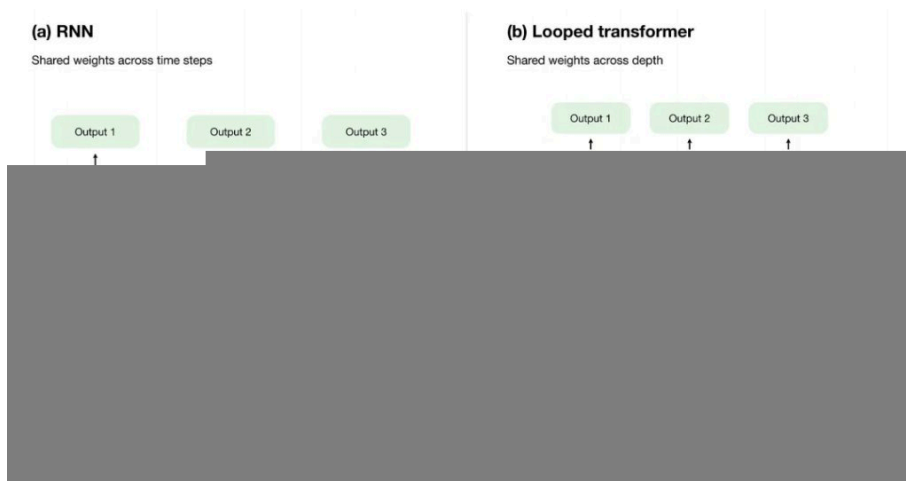


图 13：RNN 与循环 Transformer 中「循环」的并排对比。

4. Astra 到底用没用循环 Transformer？

在讨论循环 Transformer 机制是否像前面 The Information 那段引述所暗示的那样遮蔽了推理轨迹之前，先问一个更基本的问题：GPT-6 Astra 究竟有没有用循环 Transformer 这套东西？

图 14：来自 The Information 的引述，来源：<https://www.theinformation.com/articles/secret-technique-behind-openais-astra-model-sparks-security-concerns>

我们必须记住，这目前仍然只是传言或独家爆料，没有官方确认。如果模型是开放权重的，我们当然可以自己去核实，但眼下只能依赖未经证实的报道。

不过我认为，GPT-6 Astra 很可能确实用了循环 Transformer 的相关设计。第一，有上面提到的报道。第二，这项技术在过去的研究中已经显示出潜力（如前所述），那为什么不用呢？第三，OpenAI 的首席科学家说过下面这段话。

[.....] 我们当前前沿模型（包括 Astra）的计算图深度，与 GPT-4 的差距在两倍以内。[.....]

但这并没有明确证实循环 Transformer 架构的存在，它也可能只是意味着他们用了两倍数量的常规 Transformer 块。

在我看来，Astra 的成功（也就是良好的建模性能）主要恐怕还是来自其他原因，即更好的训练配方和训练数据。

循环 Transformer 这个改动或许有些帮助，但我认为 The Information 高估了它的贡献。

5. 隐藏思维链

接下来，终于要面对那头房间里的大象了：循环 Transformer 会遮蔽推理轨迹吗？

首先，OpenAI 从一开始，至少从 OpenAI o1 起，就一直对用户隐藏大部分推理轨迹。所以对终端用户来说，这里不该有太大差别。

因此，可解释性方面的担忧主要是针对模型开发者的。

无论如何，我不认为循环 Transformer 是隐藏或遮蔽思维链的主要推手。为了说清楚我的理由（无意双关），我们先退一步，讲讲推理模型是怎么工作的。

5.1 简述推理

推理模型通常会在给出最终答案之前生成一些中间步骤。这些步骤使用的是普通的文本 token（在某些用户界面中可以选择对用户隐藏），被称为推理轨迹或者思维链。

举个例子，假设我们要找两个数，和为 10、积为 21。在下图中，模型一开始试了 5 和 5。虽然和对了，但乘积是 25，不是 21。接着它试了 3 和 7，再把这两个条件重新检验一遍。

图 15：一段示意性的 LLM 回答，标注出了中间步骤、回溯和最终答案。

这张图展示了推理模型是如何「推理」的，包括回溯。也就是说，模型发现了错误，回到先前的某个选择，然后换一种思路继续。

注意，模型仍然是一次生成一个 token，并把提示词和先前的 token 作为上下文。所以这些中间步骤起到的是草稿纸的作用，在给出最终答案之前增加了计算量。

如上例所示，最终答案可能比它之前的推理轨迹短得多。（OpenAI 倾向于对用户隐藏大部分推理轨迹。）

想更深入地理解和开发推理模型，推荐我的书《Build a Reasoning Model From Scratch》。

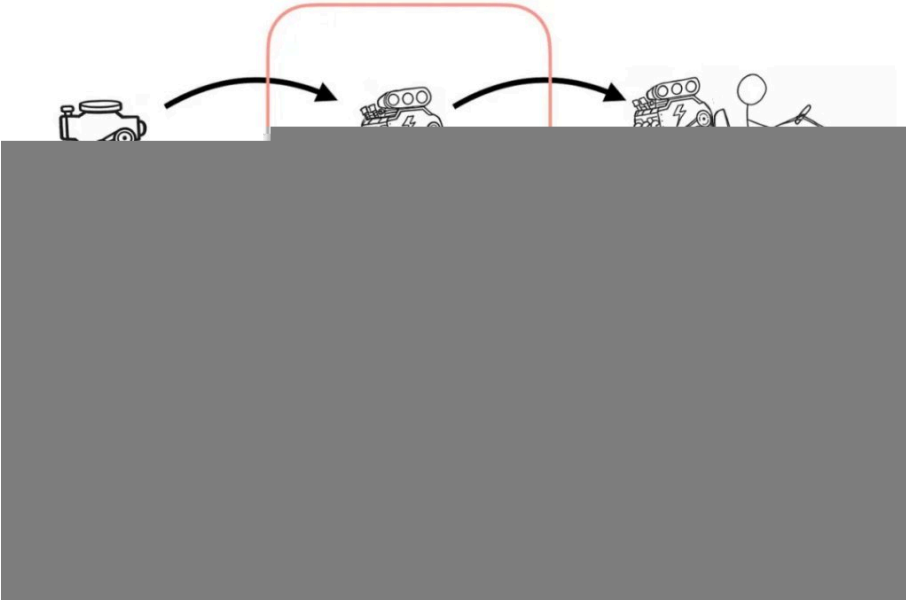


图 16：我的《Build a Reasoning Model From Scratch》一书讲解了推理模型的基础原理。

5.2 token 用量与更短的思维链

推理轨迹中多出来的 token 会带来更多计算。而循环 Transformer 同样会增加计算，因为 token 要通过更多的 Transformer 块。于是有人可能会说：既然带循环的模型在内部消耗了更多计算，那它就不需要那么多外部的思考 token 了。

下面选取了部分 GPT-6 的基准结果，x 轴是输出 token 数。

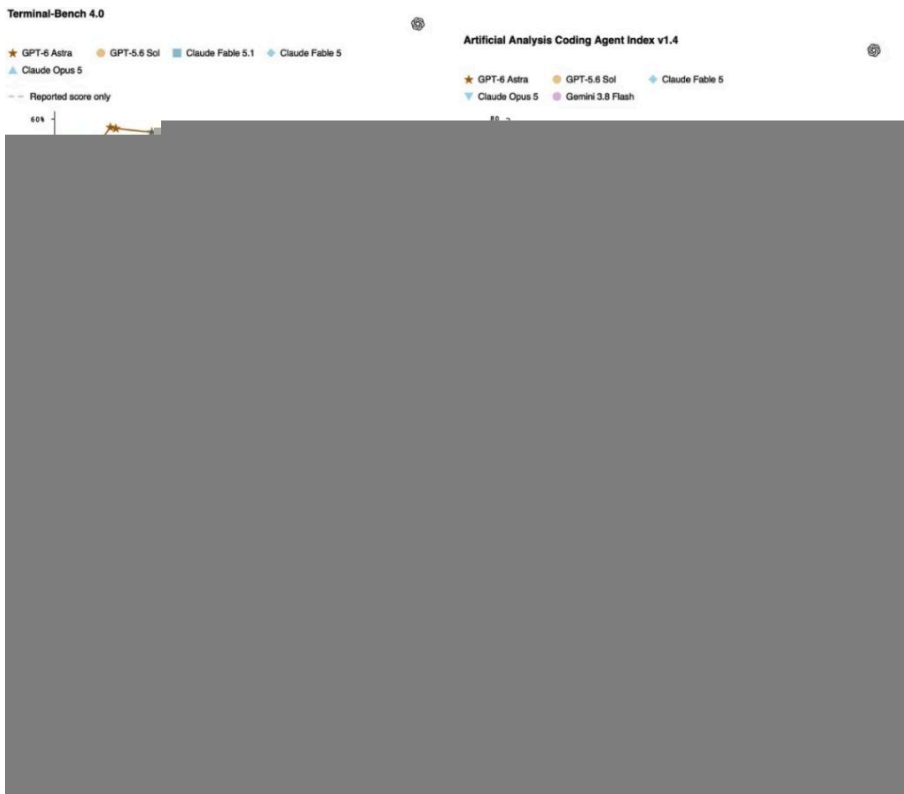


图 17：选自 <https://openai.com/index/gpt-6-astra/> 的部分 GPT-6 Astra 基准结果。

可以看到，在各个 effort 档位上，GPT-6 Astra 整体上并不一定比前代 GPT-5.6 Sol 用更少的 token。不过，在准确率固定的条件下，GPT-6 Astra 确实比 GPT-5.6 Sol 用得更少。

这对可解释性构成担忧吗？未必。token 用得更少，可能只是说明模型能力更强、犯错更少、回溯更少等等。也就是说，它可能只是更多地在第一次尝试时就做对了。在我看来，这并不会立刻引发可解释性方面的担忧。

我的意思是，以前的模型也是如此。我不觉得会有谁强烈担心 GPT-5.6 Sol 的可解释性比更小的 GPT-5.6 Luna 差很多，而如下图所示，Luna 在达到相同任务表现时要用掉多得多的 token。

Artificial Analysis Intelligence Index v4.3, weighted averages per task

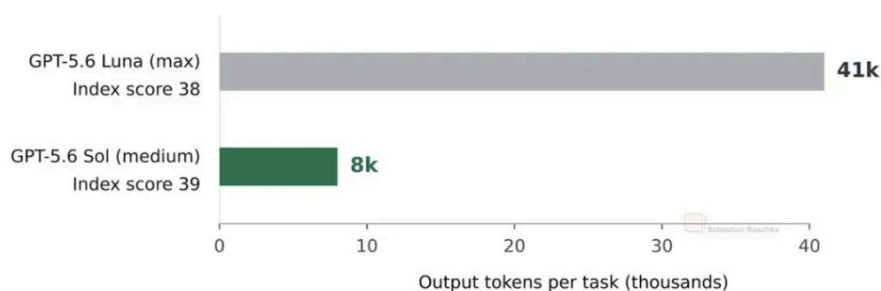


图 18：Luna 和 Sol 在相近任务表现下的 token 用量。数据来自 Artificial Intelligence Index v4.3。

事实上我们可以看到，在建模性能相近的情况下，Luna 比 Sol 多用了 80% 的 token。难道这就意味着 Sol 的可解释性差那么多吗？

更合理的解释是：能力更强的模型（更大、训练更充分、消耗更多算力）能更高效地解决问题，而这里的「高效」指的就是 token 更少

还需要记住的一点是，推理轨迹并不保证如实描述模型内部发生的一切。在我看来，唯一站得住脚的担忧是：循环 Transformer 会不会比常规 Transformer 更频繁地用「假的」推理轨迹刻意误导用户。但我认为，目前没有任何有力证据表明这种情况正在发生。

另外，Astra 的系统卡确实提到，有证据显示其推理轨迹的可监控性有所下降，相对 Sol 出现了一定程度的退步。这主要与轨迹更短、信息量更少有关。但这同样无法证明循环就是根本原因。它也可能只是因为整体长度变短了，就像上面 Luna 与 Sol 的例子一样。

在我分享了自己关于循环 Transformer 与隐藏推理链的看法几小时之后，OpenAI 首席科学家 Jakub Pachocki 也发布了如下澄清：

我想避免因混乱的报道而引发一场奔向「不可监控」的竞赛。我们当前前沿模型（包括 Astra）的计算图深度，与 GPT-4 的差距在两倍以上。自最早的推理模型起，OpenAI 就一直在努力保留并利用思维链监控。我们非常重视这项技术，因为它能让我们看到模型的对齐是如何从训练分布中泛化出去的。我确实认为它很脆弱，而且不幸的是正朝着不好的方向发展，原因与架构变化无关，我很快会专门撰文说明。但我们有办法强化它，这也是我们当前研究计划的一个核心目标。

这里说的「混乱的报道」，多半指的就是前面提到的 The Information 那段话。言下之意是，循环这件事与思维链的变化没有任何关系。

6. 循环 Transformer 相关研究

最后，我想分享几篇与循环 Transformer 架构相关、且前面尚未讨论过的有趣论文。

6.1 潜在推理

与 Universal Transformer 相关的，是 2025 年的论文《Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach》，它研究的是模型如何在推理时使用额外的循环。为此，他们在 8000 亿 token 上训练了一个规模不大、但也不算特别小的 35 亿参数模型。

它不像 Universal Transformer 那样反复复用同一个块，而是像 Nanbeige 那样重复一整个栈；但与 Nanbeige 不同的是，它把这个由 4 个块组成的共享栈，夹在 2 个起始块和 2 个末尾块之间。

与 Nanbeige 的另一处不同在于：在每一轮循环开始时，共享栈除了接收上一轮的隐藏状态，还会接收起始块的输出。两者会被拼接起来，经过一个可学习的线性投影，再进入这 4 个共享块。你可以把它理解为：让这个栈在每一遍都能拿到同样的初始输入表示。整体结构总结在下图中。

简而言之，这是循环 Transformer 的又一个有趣变体。

图 19：Geiping 等人论文中潜在推理模型的概念性总结。

一个有意思的细节是，研究者在训练过程中会改变循环次数。这让模型做好准备，以便在推理时适应不同的计算量。

具体来说，训练时的循环次数是随机采样的。而在推理时，则由运行模型的人选定一个固定预算，比如 8 次、32 次或 64 次循环。此外，他们还还为每个 token 设计了基于下一 token 概率分

布的自适应停止机制。如果连续两轮之间的 KL 散度低于某个阈值，也就是两个分布过于相似，循环就会停止。

总体收益取决于任务。在他们的评测中，HellaSwag 的表现在大约 8 轮之后基本趋于平缓，而 GSM8K 和 HumanEval 则能从更多轮次中获益。

不过，虽然论文标题里提到了「潜在推理」，模型仍然可以生成文本形式的思维链。循环只是在输出每个 token 之前给它更多计算。

6.2 知识检索 vs 推理

「存储信息」和「用信息解决问题」之间有一个很有用的区分。比如 2025 年 6 月的论文《Beyond Parameters: Exploring Virtual Logic Depth for Scaling Laws》，就通过分别测量 LLM 的记忆能力和推理能力来研究这一点。

首先，在记忆实验中，当参数量固定时，循环几乎不改变模型存储的信息量。而增加不同参数的数量，确实会提升这一容量。由此可以得出结论：循环并不会增加知识，也不会让模型检索到更多知识。这也说得通。一旦信息已经存下来了，检索本身是相对简单的任务。而且，循环本身是一种计算机制，而非「存储」机制。

其次，在另外的推理实验中，复用这些块能在不增加参数的情况下提升多步数学题的表现。这就有意思了。这里可以得出的结论是：即便模型没有更多空间来存储信息，额外的计算也能帮它解决问题。但话说回来，更大的模型同样能提升推理能力，只不过它也增加了参数。



图 20：在这项记忆测试中，容量随参数量增长，但增加块的应用次数几乎不改变容量。图片出自 Zhu 等人的论文，并加了标注。

6.3 在算力预算对齐条件下的循环

刚刚发布的 2026 年 9 月论文《SMELT: Scaling Laws for Compute-Matched MoE Looped Transformers》，重新回到了 2.2 节的成本比较问题。如果我们在每 token 计算量、非嵌入总参数量和 KV 缓存需求都大致相同的条件下，去比较循环 Transformer 和常规 Transformer，会发生什么？

研究者采用混合专家架构，把中间那一半的 Transformer 块应用两次。这做法有点像 Nanbeige，只不过用上了 Latent Reasoning 那种「夹心」结构

不过，为了抵消额外块应用带来的计算开销，他们收窄了隐藏维度。而这又会让参数量变小，于是他们再增加专家数量，把总参数量补回来。他们还调整了注意力头的配置，以保持 KV 缓存规模可比。

图 21：SMELT 概览，取自 SMELT 论文 3.2 节给出的示例。

实验规模最高做到了 540 亿非嵌入参数等等。随后，研究者根据拟合出的扩展曲线估计：在所研究的算力区间内，SMELT 达到同样的验证损失所需的训练算力大约少 6.8% 到 18%。

所以，这就回答了「循环 Transformer 在计算上值不值」这个问题：值！在相同算力预算下，它们能给我们一个稍好一些模型。

6.4 全带宽 Transformer

最后，同样非常新的 2026 年 8 月论文《Full-bandwidth transformer》研究的是跨 token 位置的循环。在每一个解码步骤，它通过一个可学习的门控，把上一个 token 的最终隐藏状态与新采样出的 token 的嵌入结合起来。这个结果会成为下一次前向传播的输入。

于是，下一个 token 的计算从栈的最底部就能拿到上一个 token 的最终表示，这在某种程度上与 Latent Reasoning 相似。

在使用 1B 基座模型时，他们发现这种潜在反馈方法在 MATH500 上会输出更短的推理轨迹，同时保持甚至提升了准确率。不过，经过指令微调之后，这种缩短效应就消失了。

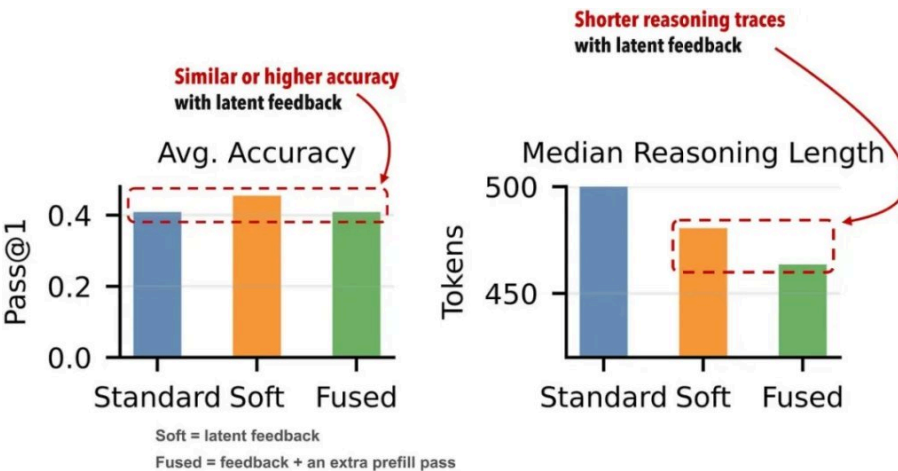


图 22：潜在反馈缩短了基座模型的推理轨迹，但这一效应在指令微调后消失。改编自 Wang 等人论文图 6，CC BY 4.0。定义和注意事项为作者补充。

总之，这一点很有意思，因为它直接呼应了前面关于「循环是否会带来更短推理轨迹」的讨论。当然，结果同时取决于反馈机制和模型的训练方式。另外，该实验也没有确定这些更短的轨迹是否忠实度更低。

此外，这项研究的一大局限在于：他们没有测试用常规方式增大模型（也就是堆更多 Transformer 块而不是循环）是否会对推理轨迹长度产生类似的影响。

结语

总结一下。我们可以说，OpenAI 的 GPT-6 Astra 确实是个非常强的模型，而且它在计算机操作方面迈出了特别大的一步。我相信在接下来的几个月里，计算机操作会成为开源和闭源框架的

下一个重点方向。在计算机操作这件事上，我认为开源尤其重要，毕竟「能力越大，责任越大」，在把主力电脑的访问权限交出去之前，能先审计一下这个框架是件好事。

此外，GPT-6 Astra 很可能用了某种循环 Transformer 的变体。循环 Transformer 就是能在固定算力预算下带来更好的建模性能。

还有，更好的建模性能可能会伴随着更短的推理链。但这并不是什么新趋势。在同一模型家族里不同规模的模型之间（比如 GPT-5.6 Luna 与 Sol），我们一直都能看到这一点。

在我看来，更短的推理轨迹是更「聪明」、能力更强的模型带来的副产品。这类模型犯错更少，而且能在架构内部调动更多计算，而不是把推理轨迹当草稿纸用。某种意义上，人也是如此。在大学的现场数学考试中，一个聪明且准备充分的学生，大概率更少用到草稿纸，也更少需要回头重来。

本文来源：[机器之心](#)

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

 限时权益申领

* 体验资格每人限申领一次

扫码 

免费领取

VIP会员·7天畅读卡 | 大师课·入门串讲



限免

 79  收藏

分享到:  

写评论

请发表您的评论

 表情  图片

发布评论